

PROJECT REPORT

CS-2009 Design and Analysis of Algorithms

Abdul Mueed Malik I221622
Nameer Khan I221695



National University
of computer and emerging sciences

FAST NUCES, Islamabad

Department of Cyber Security

1. Introduction.....	3
Objective.....	3
Scope.....	3
2. Problem Definition.....	3
1. Graph-Based Analysis.....	3
2. Dynamic Programming on Graph.....	3
3. Dataset Description.....	4
4. Methodology.....	4
4.1 Graph-Based Analysis.....	4
4.2 Dynamic Programming on Graph.....	4
5. Algorithm Design.....	5
5.1 Graph Construction Algorithm.....	5
Pseudocode.....	5
5.2 Shortest Path Algorithm (Dijkstra).....	6
Pseudocode.....	6
5.3 Longest Chain of Influence (Dynamic Programming).....	7
Pseudocode.....	7
6. Time Complexity Analysis.....	8
1. Graph Loading.....	8
2. Influence and Label Loading.....	9
(a) Influence Loading.....	9
(b) Node Labels Loading.....	9
3. Dijkstra's Algorithm.....	10
4. A* Algorithm.....	11
5. Longest Influence Chain.....	11
7. Results and Discussion.....	13
Shortest Path (Dijkstra).....	13
Longest Chain of Influence.....	13
8. Full Output.....	14
9. Conclusion and Future Work.....	14
10. References.....	15

1. Introduction

Objective

The primary objective of this project is to analyze a social network to identify significant connections between users and compute the longest chain of influence using graph-based algorithms and dynamic programming techniques.

Scope

The scope includes:

- Graph construction from a dataset of user connections.
- Shortest path analysis using Dijkstra's or A* algorithm.
- Dynamic programming to compute the longest chain of influence in the network.

2. Problem Definition

The project has two major objectives:

1. Graph-Based Analysis

- Identify significant user connections by constructing an undirected graph from a dataset.
 - Compute the shortest path between users using Dijkstra's or A* algorithm.

2. Dynamic Programming on Graph

- Compute the longest chain of influence in the network.
- This chain is defined as a sequence of users where the influence score increases at each step.

3. Dataset Description

Source : [Link To Dataset.](#)

The dataset is loaded from three files:

1. Connections File: Contains 3 columns (User1, User2, Weight) representing connections and the weights (distances) between them.
2. Influences File: Contains 2 columns (User, Influence Score) representing the influence of each user.
3. Labels File: Contains 2 columns (Node, User Name) linking nodes to user names.

4. Methodology

4.1 Graph-Based Analysis

1. Graph Construction: Create an undirected, weighted graph where nodes are users and edges are connections.
2. Shortest Path Calculation: Use Dijkstra's or A* algorithm to compute the shortest path.

4.2 Dynamic Programming on Graph

1. Longest Chain of Influence: Identify the longest increasing chain of influence scores for users.
2. Dynamic Programming Approach: Develop a DP algorithm to compute the chain and return its length and sequence of users.

5. Algorithm Design

5.1 Graph Construction Algorithm

Pseudocode

Input: Connections File with columns (User1, User2, Weight)

Output: Undirected weighted graph $G(V, E)$

Procedure:

1. Initialize an empty graph G
2. For each line in the file:
 - a. Extract User1, User2, and Weight
 - b. Add an edge between User1 and User2 with weight = Weight in G

Return G

Code

```
void loadGraph(const string &filename)
{
    cout << "===== " << endl;
    cout << endl;
    cout << CYAN << "      Loading graph" << RESET << endl;
    ifstream file(filename);
    if (!file.is_open())
    {
        cerr << RED << " Unable to open file: " << filename << RESET << endl;
        return;
    }

    int u, v, w;
    while (file >> u >> v >> w)
    {
        graph[u].push_back({v, w});
        graph[v].push_back({u, w}); // Since the graph is undirected so pushing the edge in reverse direction
    }
    file.close();
    cout << GREEN << "  Graph loaded successfully" << RESET << endl;
    cout << endl;
    cout << "===== " << endl;
}
```

5.2 Shortest Path Algorithm (Dijkstra)

Pseudocode

Input: Graph $G(V, E)$, Source node S , Target node T

Output: Shortest path from S to T

Procedure:

1. Initialize distance array $dist[]$ with ∞ for all nodes except source S ($dist[S] = 0$)
2. Initialize priority queue Q with $(dist[S], S)$
3. While Q is not empty:
 - a. Extract node u with min $dist[u]$
 - b. For each neighbor v of u :
 - i. Calculate new distance $d = dist[u] + weight(u, v)$
 - ii. If $d < dist[v]$, update $dist[v]$ and add v to Q

Return $dist[T]$

Code

```
vector<int> dijkstra(int start, int end, int &totalDistance)
{
    unordered_map<int, int> dist;
    unordered_map<int, int> parent;

    for (auto &node : graph)
    {
        dist[node.first] = INT_MAX;
    }
    dist[start] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    pq.push({0, start});

    while (!pq.empty())
    {
        int current = pq.top().second;
        int currentDist = pq.top().first;
        pq.pop();

        if (current == end)
            break;

        for (auto &edge : graph[current])
        {
            int neighbor = edge.target;
            int weight = edge.weight;
            if (currentDist + weight < dist[neighbor])
            {
                dist[neighbor] = currentDist + weight;
                pq.push({dist[neighbor], neighbor});
            }
        }
    }
    totalDistance = dist[end];
    return dist[end];
}
```

5.3 Longest Chain of Influence (Dynamic Programming)

Pseudocode

Input: Graph $G(V, E)$, Influence scores for users

Output: Longest increasing influence chain

Procedure:

1. Sort users by influence score
2. For each user i :
 - a. Initialize $dp[i] = 1$
 - b. For each neighbor j of i where $influence(j) > influence(i)$:
 - i. Update $dp[j] = \max(dp[j], dp[i] + 1)$

Return $\max(dp)$ and the corresponding chain of users

Code

```
void longestInfluencePath(int startNode = -1, int endNode = -1)
{
    vector<int> nodes;
    for (auto &node : influences)
    {
        nodes.push_back(node.first);
    }

    // Sort nodes by influence in ascending order
    sort(nodes.begin(), nodes.end(), [](int a, int b)
        { return influences[a] < influences[b]; });

    unordered_map<int, int> MemoryArr; /*An map used to store Longest path length ending
    at each node to avoid repeated checks and make use of previous processing to reduce complexity.*/
    unordered_map<int, int> parent;    // Parent map for path reconstruction

    for (int node : nodes)
    {
        MemoryArr[node] = 1; // Each node can at least be a path of length 1

        for (int i = 0; i < graph[node].size(); i++)
        {
            int neighbor = graph[node][i].target;
```

6. Time Complexity Analysis

This section analyzes the time complexity of key functions from the provided C++ code. We will analyze the following parts:

1. Graph Loading
2. Influence and Label Loading
3. Dijkstra's Algorithm
4. A* Algorithm
5. Longest Influence Chain

1. Graph Loading

Function: loadGraph(filename)

Process:

- Read edges (u,v,w) from a file and populate the adjacency list for an undirected graph.
- Each edge is stored twice: from $u \rightarrow v$ and $v \rightarrow u$.

Time Complexity Analysis:

- **File Reading:** Reads E edges from the file, so file I/O takes $O(E)$ time.
- **Edge Insertion:** Each edge requires 2 insertions into the adjacency list (unordered map), which takes $O(1)$ on average.

Total Time: $O(E)$ (where E is the number of edges)

2. Influence and Label Loading

(a) Influence Loading

Function: loadInfluences(filename)

Process:

- Reads the influence value for each node from a file and stores it in an unordered map.

Time Complexity Analysis:

- **File Reading:** Reads V nodes (one influence per node) from the file, taking $O(V)$ time.
- **Insertion:** Each entry is stored in an unordered map in $O(1)$ time.

Total Time: $O(V)$ \text{Total Time:} \quad O(V)

(b) Node Labels Loading

Function: loadNodeLabels(filename)

Process:

- Reads each line, splits it into a node number and its label, and stores it in a map.

Time Complexity Analysis:

- **File Reading:** Reads V node-label pairs from the file, taking $O(V)$ time.
- **Insertion:** Each label is inserted into an unordered map, taking $O(1)$ time.

Total Time: $O(V)$ \text{Total Time:} \quad O(V)

3. Dijkstra's Algorithm

Function: dijkstra(start, end)

Process:

- Uses a priority queue (min-heap) to explore the shortest path from start to end.
- For each node, it checks its neighbors and updates the shortest distance.
- Each time a shorter path is found, it updates the distance and pushes it into the priority queue.

Time Complexity Analysis:

- **Distance Initialization:** Initializes distances for V nodes — $O(V)$.
- **Priority Queue Operations:** Each insertion or update in the priority queue takes $O(\log V)$.
- **Edge Relaxation:** Each edge is processed at most once, so total edge relaxations take $O(E)$ time.

The overall time complexity is:

Total Time: $O((V+E) \cdot \log V)$ $\text{Total Time:} \quad O((V + E) \cdot \log V)$

Time: $O((V+E) \cdot \log V)$

This complexity comes from V insertions plus E edge relaxations, where each operation on the heap takes $O(\log V)$ time.

4. A* Algorithm

Function: aStar(graph, start, end, totalWeight)

Process:

- Similar to Dijkstra, but with an added heuristic to prioritize nodes.

- Priority is determined by $f(n) = g(n) + h(n)$
 $f(n) = g(n) + h(n)$, where:

- $g(n)$ = distance from start to node n .
- $h(n)$ = heuristic value (number of neighbors).

Time Complexity Analysis:

- **Distance and Heuristic Initialization:** For each of V nodes, initialize the distance and heuristic value.

$O(V)$

- **Priority Queue Operations:** Similar to Dijkstra, each insertion or update takes $O(\log V)$.

- **Edge Relaxation:** Each edge is processed once, so total edge relaxations take $O(E)$ time.

Total Time: $O((V+E) \cdot \log V)$

Time: $O((V+E) \cdot \log V)$

This is the same as Dijkstra's complexity. However, with a good heuristic, the actual time may be significantly reduced.

5. Longest Influence Chain

Function: longestInfluencePath(start, end)

Process:

1. **Sort Nodes by Influence:** Nodes are sorted based on their influence values.

2. Dynamic Programming: Uses a bottom-up approach to find the longest path ending at each node.

3. Path Reconstruction: Reconstructs the longest path if start and end nodes are specified.

Time Complexity Analysis:

1. **Sorting:** Sort V nodes by their influence scores.

$O(V \log V)$

2. Longest Path Calculation:

- For each node, we process all its neighbors to update the longest path.

- Each edge is processed at most once, so the total work done is $O(E)$.

3. Path Reconstruction:

- Traces the path from start to end using a parent map.

- In the worst case, it processes V nodes.

Total Time: $O(V \log V + E)$

If the graph is sparse (i.e., $E \approx V$), this becomes:

$O(V \log V)$

7. Results and Discussion

Shortest Path (Dijkstra)

Input: Start Node = 5, End Node = 6

Output:

Shortest Path:

```
=====
Enter The Start Node: 6595
Enter The End Node: 1392
=====

Shortest Path (Dijkstra)
From : 6595
To : 1392

Shortest Path: 6595 (Trevor Jones) -> 14063 (Lorraine Hopkins) -> 3243 (Jesse Welch) -> 21070 (Vincent Bridges Jr.) -> 1389 (Jonathon Walker) -> 1392 (Shelia Huffman)
Distance: 88
Execution Time (Dijkstra): 107909 microseconds
```

Longest Chain of Influence

Input: Whole Graph (40,000 nodes)

Output:

```
=====
PART 2
Select Longest Influence Chain:
1 . Between Two Nodes
2. Whole Graph

Longest Increasing Influence Path:

Length: 18

Chain: 6595 (Trevor Jones) -> 14063 (Lorraine Hopkins) -> 3243 (Jesse Welch) -> 21070 (Vincent Bridges Jr.) -> 1389 (Jonathon Walker) -> 1392 (Shelia Huffman) -> 21445 (Katherine Wood) -> 26187 (Rhonda Cox) -> 9799 (Christy Schwartz) -> 212 (Jason Baker) -> 28661 (Tracy Olson) -> 11490 (Susan Lopez) -> 20021 (Eric Underwood) -> 35737 (Kimberly Roberts) -> 21394 (Cory Anderson) -> 9318 (Mrs. Carolyn Baker) -> 20614 (Joshua Gray) -> 1860 (Sara Conner)

PS C:\Users\Abdul Mueed\Desktop\algo rpoj> |
```

8. Full Output

```
PS C:\Users\Abdul Mueed\Desktop\algo_rpoj> ./social_proj
=====
Loading graph
Graph loaded successfully

=====

Loading Influences
Total Nodes: 40000
Influences Loading Completed

=====

Loading Labels
Labels Loading Completed

=====

Enter The Start Node: 4
Enter The End Node: 6

=====

Shortest Path (Dijkstra)
From : 4
To : 6

Shortest Path: 4 (Devon Gallagher) -> 37743 (Karen Erickson) -> 4052 (Nicole Price) -> 5963 (Charles Martin) -> 11735 (Andres Farmer) -> 5
487 (Jessica Baker) -> 23214 (Jeffrey Nichols) -> 11066 (Parker McCormick) -> 22808 (Caitlin Powell) -> 8473 (Amy Richmond) -> 6 (Michael
Kim)
Distance: 97
Execution Time (Dijkstra): 138011 microseconds

Shortest Path (A*)
From : 4
To : 6

A* Path: 4 (Devon Gallagher) -> 37743 (Karen Erickson) -> 4052 (Nicole Price) -> 5963 (Charles Martin) -> 11735 (Andres Farmer) -> 5487 (J
ctrl + k to regenerate content
```

9. Conclusion and Future Work

Conclusion

- We successfully constructed a graph with 40,000 nodes.
- We computed the shortest path between two nodes using Dijkstra's algorithm.
- We computed the longest increasing chain of influence using dynamic programming.

10. References

- Introduction to Algorithms, Cormen et al.
- Data Structures and Algorithms in C++, Mark Allen Weiss.

